(a) Statistical power AUC for detecting quantized variants. **Bold** = best method; gray = none reliable.

Model	RUT	MMD	KS
Gemma–4bit	**0.392**	0.214	0.017
Gemma–8bit	0.049	0.043	0.001
Llama–4bit	**0.642**	0.625	0.474
Llama–8bit	0.132	**0.158**	0.005
Mistral–4bit	**0.586**	0.500	0.330
Mistral–8bit	0.049	0.090	0.006

(b) Statistical power AUC for detecting jail-breaking prompts. **Bold** = most effective method per prompt.

Model	Prompt	RUT	MMD	KS
Mistral	Dan	**0.895**	0.802	0.873
	Anti-Dan	**0.893**	0.781	0.872
	Evil-Bot	**0.892**	0.766	0.873
Gemma	Dan	**0.888**	0.757	0.867
	Anti-Dan	**0.858**	0.816	0.854
	Evil-Bot	**0.893**	0.753	0.871

Table 2: Side-by-side comparison of statistical-power AUCs. Left: quantized variants; right: jail-breaking prompts.

Baseline. For the detection methods (Sun et al., 2025; Gao et al., 2025) that are compatible with WildChat, We primarily focus on Maximum Mean Discrepancy (MMD) (Gao et al., 2025) as the baseline, as Sun et al. (2025) is reported to fail to identify quantization (Cai et al., 2025).

We also tailor a Kolmogorov–Smirnov (KS) test baseline that uses the same information as RUT: it computes the log-rank scores from the reference model on both the target and reference model responses and applies the two-sample KS test (Darling, 1957) on these two sets of scores to estimate the p-value between the target and reference distributions.

Test procedures. We apply a consistent sample budget constraint on all tests. The implementation details of their test procedures are listed below:

- **Rank-Based Uniformity Test (RUT):** Each trial samples 100 prompts. We query each prompt once to the target and 100 times to the reference model.
- **Maximum Mean Discrepancy (MMD):** We apply the MMD test based on the character-level Hamming distance following Gao et al. (2025). Each trial uses 10 prompts, with 10 samples per prompt. We compute the MMD statistic and estimate the p-value via 500 random permutations.
- **Kolmogorov–Smirnov Test (KS):** We use the same sampling setup as RUT: 100 prompts per trial, 1 query to the target, and 100 to the reference model per prompt.

Across all models, we set the temperature to 0.5 and cap generation at 30 tokens. We use vLLM (Kwon et al., 2023) on a single A6000 for all local inferences.

To estimate statistical power, we repeat each experiment over 500 Monte Carlo trials. Power is computed as the proportion of trials in which the test correctly rejects the null hypothesis at a significant level of 0.05. All statistical tests were run with Intel Xeon Gold 6230R @ 2.10GHz and 16 GB RAM per job.

5.2 Detecting Quantization

We consider the setting where the API provider uses a quantized variant to substitute the claimed model. We evaluate three detection methods on quantized variants of Llama-3.2-3B-Instruct[1], Mistral-7B-Instruct-v0.3[2], and Gemma-2-9B-it[3], comparing each model to its 4-bit and 8-bit quantized counterparts. As shown in Table 2a, none of the methods succeed in reliably detecting substitution for the 8-bit variants of Gemma and Mistral, where statistical power AUC remains near zero across the board. In the remaining four settings, RUT outperforms MMD and the KS baseline in 3 out of the 4 cases, demonstrating superior sensitivity to quantization-induced distributional shifts. Full statistical power curves for AUCs are provided in Appendix B.1.

5.3 Detecting Jailbreaks

We consider the setting where the API provider secretly appends a hidden jailbreaking system prompt to user queries. To evaluate this scenario, we use two base models: Mistral-7B-Instruct-v0.3 and

[1] https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct
[2] https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3
[3] https://huggingface.co/google/gemma-2-9b-it

Gemma-2-9B-it. For each model, we construct a test using three representative jailbreaking prompts *Dan*, *Anti-Dan*, and *Evil-Bot* adapted from Shen et al. (2024).

As shown in Table 2b, all six jailbreak cases are reliably detected, with statistical power AUC values consistently above 0.75. RUT achieves the highest power in all 6 settings, demonstrating its superior sensitivity to model deviations caused by hidden jailbreaking prompts. Full statistical power curves for AUCs are provided in Appendix B.2.

5.4 Detecting SFT

We study the setting where the API provider fine-tunes a model on instruction-following data. Specifically, we fine-tune two base models—Llama-3.2-3B-Instruct and Mistral-7B-Instruct-v0.3—on benign and harmful instruction-following datasets. We use Alpaca (Taori et al., 2023) as the benign dataset and BeaverTails (Ji et al., 2023) for harmful question answering. Each model is fine-tuned on 500 samples from the respective dataset for 5 epochs using LoRA (Hu et al., 2021) with rank 64 and $\alpha = 16$, a batch size of 32, and a learning rate of 1×10^{-4} on a single A100. For each checkpoint, we compute the statistical power AUC of the detection methods.

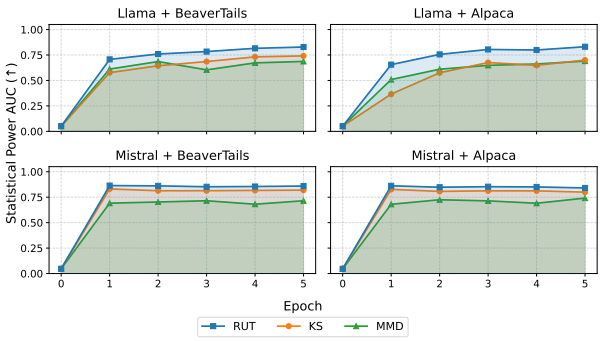

Figure 3: Statistical power AUC across epochs for detecting SFT model substitution.

As shown in Figure 3, RUT consistently achieves higher statistical power AUC than both the KS and MMD baselines across all fine-tuning configurations. Notably, our method detects behavioral changes within the first epoch of fine-tuning, demonstrating strong sensitivity to early-stage distributional shifts. While all methods improve with additional training, RUT remains the most robust across both models and datasets. Full statistical power curves for AUCs are provided in Appendix B.3.

5.5 Detecting Full Model Replacement

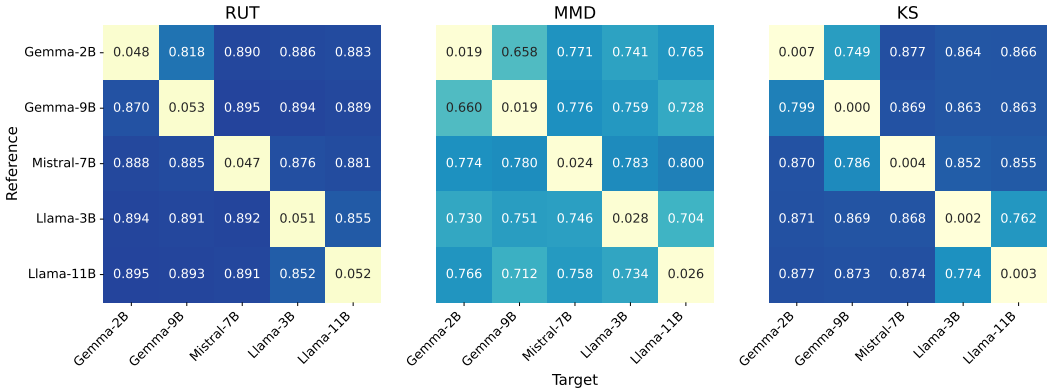

Figure 4: Statistical power AUC for detecting full model replacement. Each cell shows the AUC score between a reference and a target model. Diagonal values represent self-comparisons.

We evaluate the setting where the API provider substitutes the claimed model with a completely different one. To simulate this scenario, we conduct pairwise comparisons among five open-source models: Llama-3.2-3B-Instruct, Llama-3.2-11B-Vision-Instruct[4], Mistral-7B-Instruct-v0.3, Gemma-

[4] https://huggingface.co/meta-llama/Llama-3.2-11B-Vision-Instruct

2-2B-it[5], and Gemma-2-9B-it. For each pair, one model serves as the reference model while the other acts as the deployed target model. As shown in Figure 4, RUT consistently achieves the highest statistical power AUC across model pairs, outperforming both the MMD and KS baselines. The results highlight the method's sensitivity to full model substitutions. Full statistical power curves for AUCs are provided in Appendix B.4.

5.6 Detecting Real API Providers

Table 3: Statistical power for detecting differences from the baseline model deployed on an A6000 GPU. `A100` denotes the same model run locally on an A100 GPU; other entries are actual API providers. Values > 0.5 indicate significant behavioral deviation. Green = no significant difference; Red = significant difference.

Model	Provider	RUT	MMD	KS
Llama	A100	0.094	0.142	0.002
Llama	Nebius	0.962	0.944	0.426
Llama	Novita	0.988	0.996	0.530
Mistral	A100	0.058	0.138	0.004
Mistral	HF Inference	0.188	1.000	0.000
Gemma	A100	0.060	0.084	0.000
Gemma	Nebius	0.312	0.432	0.008

We evaluate our detection methods on three base models—Llama-3.2-3B-Instruct, Mistral-7B-Instruct-v0.3, and Gemma-2-9B-it—each deployed through multiple API providers. Local inference on an A100 GPU serves as the baseline. As shown in Table 3, all tests correctly identify behavioral equivalence in local deployments.

Across all settings, RUT and MMD generally agree in detecting significant deviations across providers, offering mutual validation for their behavioral sensitivity. The KS test exhibits similar trends but with notably lower sensitivity.

An exception arises in the Mistral + HF Inference setting, where MMD yields a power of 1.000 while other tests are below 0.2. Upon investigation, we suspect that the discrepancy is due to a tokenization mismatch: the Hugging Face Inference API consistently omits the leading whitespace present in the reference outputs. Because MMD uses character-level Hamming distance, this formatting difference inflates the score. After restoring the missing space, the MMD score drops to 0.211, aligning with other tests. This illustrates RUTt's robustness to minor decoding mismatches that can mislead string-based metrics.

6 Conclusion

The stable increase in the size (Kaplan et al., 2020) and architectural complexity (Zhou et al., 2022) of frontier LLMs has led to a rise in the popularity of API-based model access. To prevent performance degradation and security risks from model substitution behind API interfaces, this work proposes the rank-based uniformity test for model equality testing. We test the method against a variety of different substitution attacks and demonstrate its consistent effectiveness in detecting substitution and its superiority over existing methods.

Limitations and Future Work Using prompts from the WildChat dataset (Zhao et al., 2024) for testing, we aim to avoid detection of our testing attempts. However, we have not empirically validated its effectiveness at evading detection, especially against detection methods based on the correlation across prompts. In future work, examining the method's detectability and improving it with adaptive methods for prompt selection shall be a priority. Also, our test requires a locally deployed authentic model, limiting its capability to test black-box models.

By developing an effective and stealthy API-based test for model equality, we hope to advance the safety and security of LLM-based applications in the age of increasingly cloud-based deployment.

[5] `https://huggingface.co/google/gemma-2-2b-it`